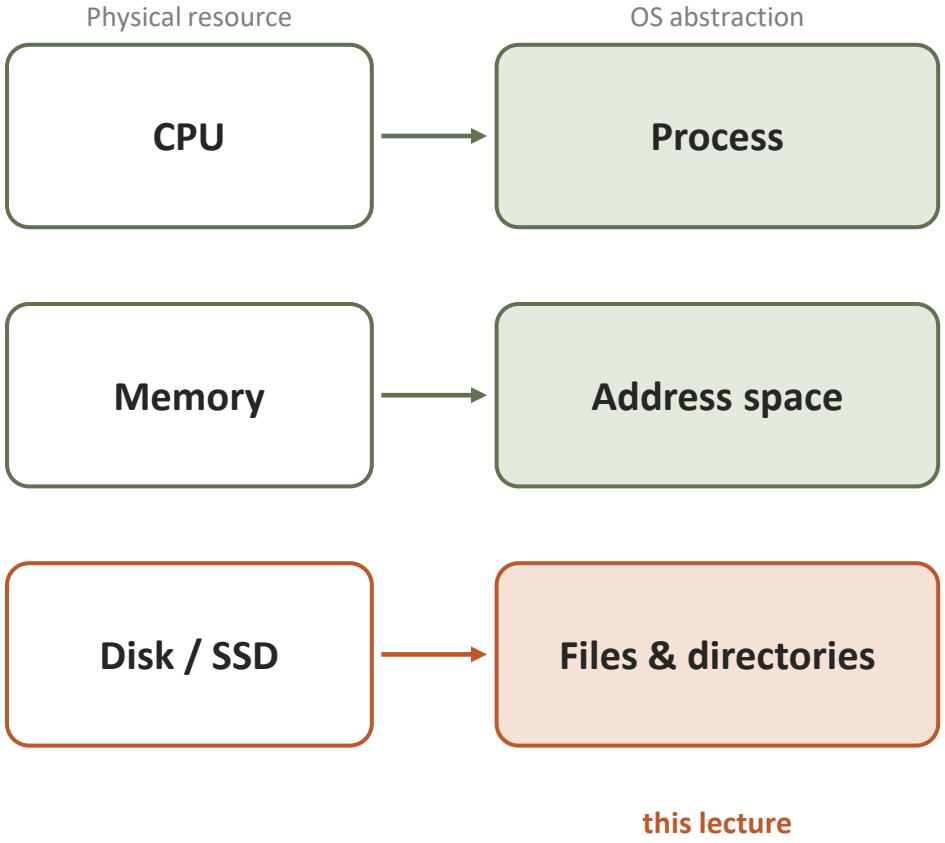
CS3.301: Operating Systems and Networks

Files and Directories

INSTRUCTOR: DR. DEEPAK GANGADHARAN

From CPU and Memory to Persistent Storage

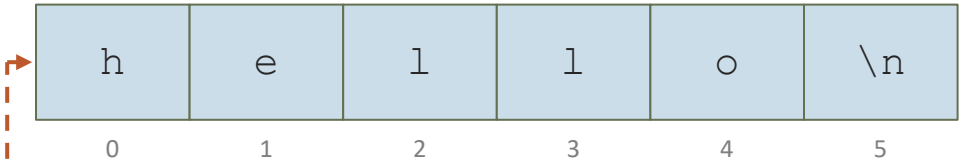
- **Process** virtualizes the CPU
- **Address space** virtualizes memory
- Next piece: **persistent storage** (HDD, SSD)
- Memory is volatile; persistent devices keep data across power loss
- Users keep data they really care about here, so the OS must take extra care
- Crux: How should the OS manage a persistent device? What are the APIs?



Two Key Abstractions: Files and Directories

- **File**: a linear array of bytes you can read or write
- Each file has a low-level name: its **inode number** (i-number)
- The OS does not interpret the contents; it stores them and returns exactly what was written
- **Directory**: also has an inode number; contents are (name, inode number) pairs
- Entries point to files or other directories, so they form a **directory tree**

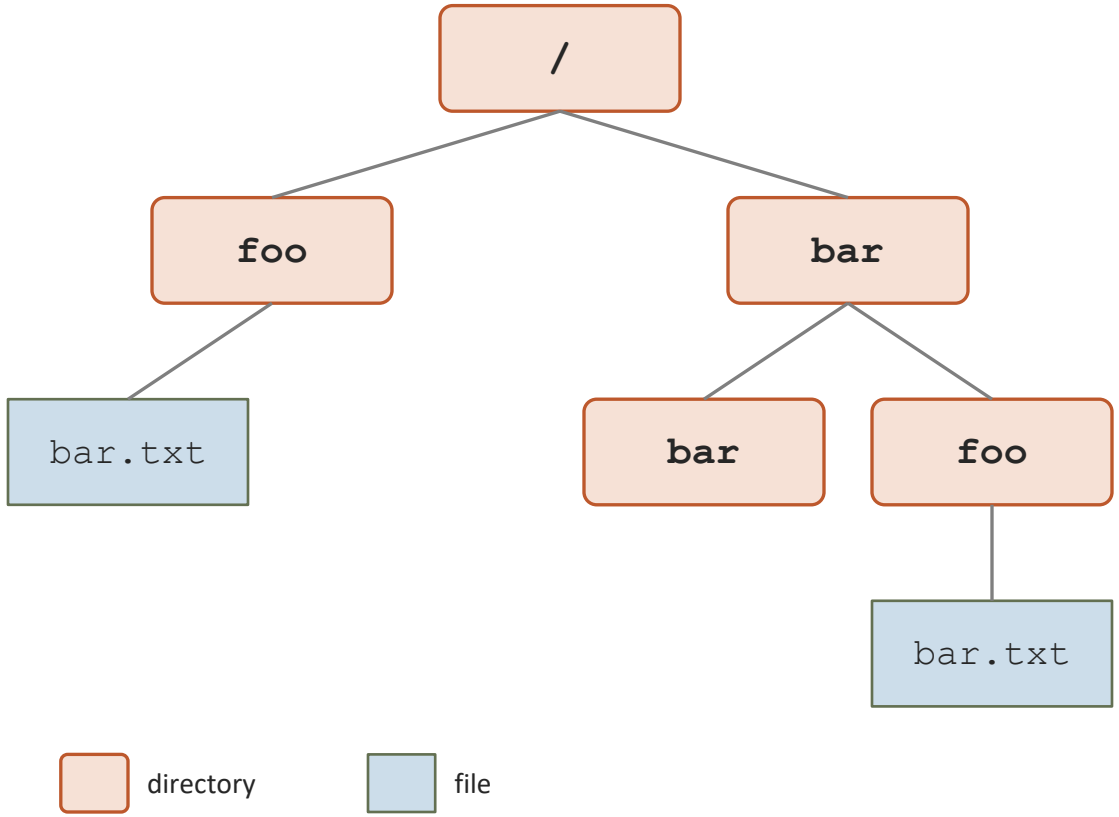
File "foo" (inode 10): an array of bytes



Directory (its own inode, e.g. 2): name → inode

User-readable name	Inode number	Refers to
`foo`	10	file
`notes.txt`	14	file
`docs`	22	directory

The Directory Tree and Pathnames



- Tree starts at the **root** `/`; `/` also separates path components
- **Absolute pathname**: `/foo/bar.txt`, `/bar/foo/bar.txt`
- Names only need to be unique within one directory
- Extensions (`.c`, `.jpg`) are a convention, not enforced
- UNIX names almost everything this way: devices, pipes, even processes (`/proc`)
- One tree gives one uniform way to name every resource

Creating Files with open()

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC, S_IRUSR | S_IWUSR);
```

O_CREAT	O_WRONLY	O_TRUNC	S_IRUSR | S_IWUSR
Create the file if it does not exist	Open for writing only	If it exists, truncate to 0 bytes (old contents gone)	Permissions: owner can read and write

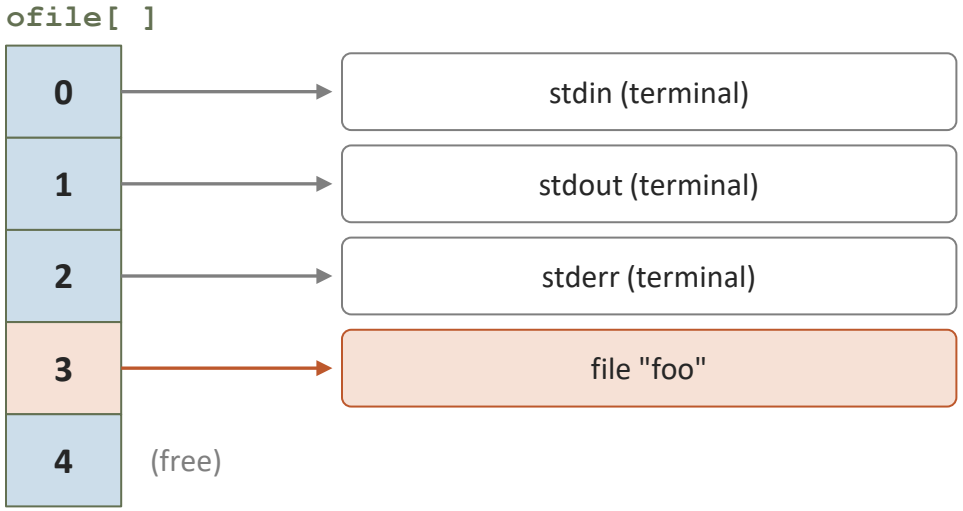
Returns a **file descriptor** (small integer)
or **-1** on error: always check it!

File Descriptors

- open() returns a **file descriptor**: an integer, **private per process**
- A **capability**: an opaque handle that lets you read(), write(), close() the file
- Like a pointer to a "file" object whose methods are system calls
- Already open in every process: 0 = stdin, 1 = stdout, 2 = stderr
- So the first file you open usually gets fd 3

xv6: per-process descriptor array

```
struct proc {
  ...
  struct file *ofile[NOFILE]; // Open files
};
```



Reading and Writing: What Does cat Do?

Shell

```
prompt> echo hello > foo
prompt> cat foo
hello
prompt>
```

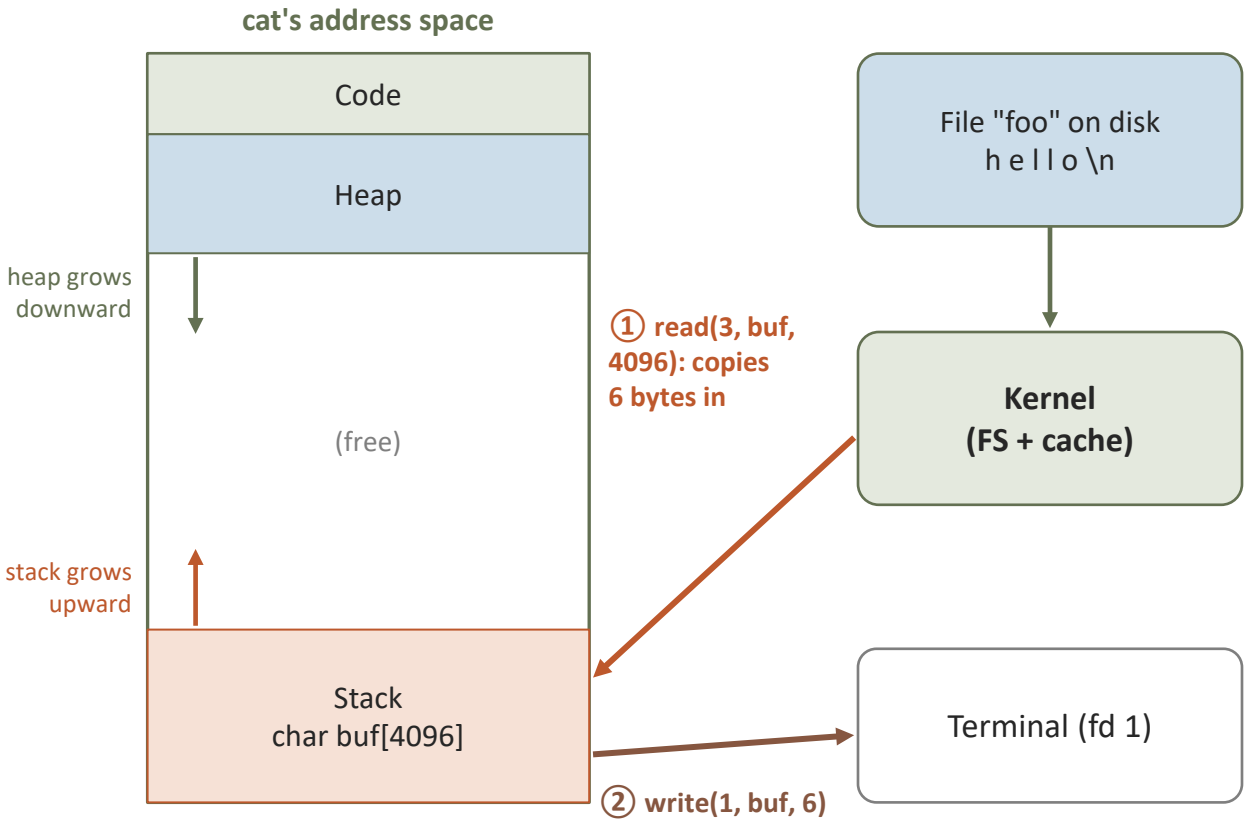
strace flags: `-f` follow children,
`-t` timestamps,
`-e trace=open,read,write,close`

strace: every system call cat makes

```
prompt> strace cat foo
...
open("foo", O_RDONLY|O_LARGEFILE) = 3
read(3, "hello\n", 4096)          = 6
write(1, "hello\n", 6)            = 6
hello
read(3, "", 4096)                 = 0
close(3)                          = 0
...
```

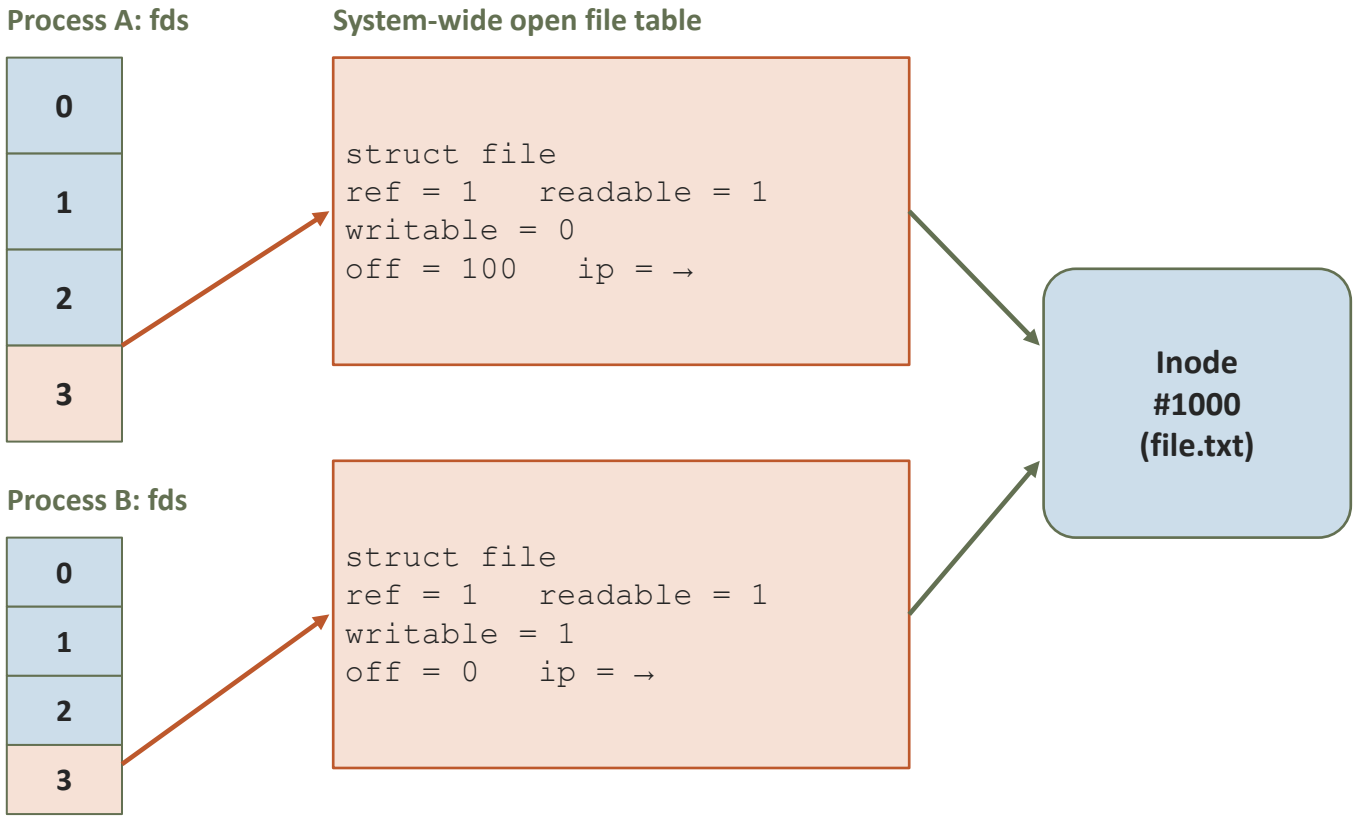
1. open → fd 3
2. read 6 bytes
3. write to fd 1
4. read → 0 (EOF)
5. close(3)

Following the Data Through read() and write()



- `read(fd, buf, count)`: which file, where to put the bytes, buffer size
- Returns bytes actually read: 6 = 5 letters + newline
- Returns **0** at end of file, so the program knows it is done
- `write(1, buf, 6)` sends the bytes to stdout (maybe via printf())
- Writing a file: open for writing → write() repeatedly → close()

The Open File Table



xv6 (simplified)

```
struct file {
  int ref;
  char readable;
  char writable;
  struct inode *ip;
  uint off;
};
```

The whole table: one array, one lock

```
struct {
  struct spinlock lock;
  struct file file[NFILE];
} ftable;
```

Offset, access mode and refcount live in the **table entry**, not in the fd

Random Access with lseek()

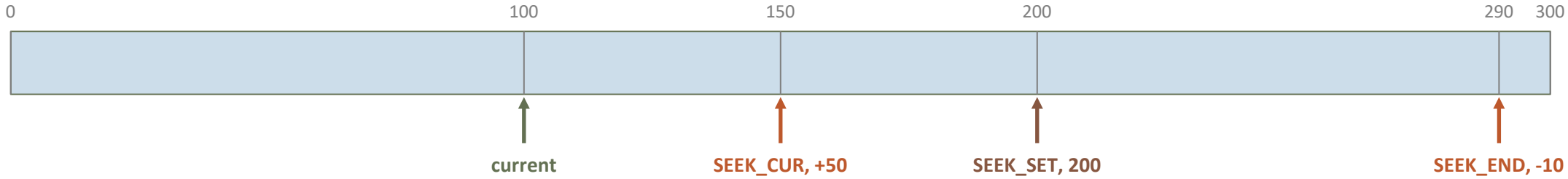
```
off_t lseek(int fildes, off_t offset, int whence);
```

whence	New offset becomes
SEEK_SET	offset
SEEK_CUR	current position + offset
SEEK_END	file size + offset

lseek() does NOT perform a disk seek.
It only changes the offset variable in OS memory; no I/O happens.

Offset also advances implicitly: a read/write of N bytes adds N

300-byte file, current offset = 100



Tracing the Current Offset

Sequential reads of a 300-byte file

System call	Returns	Offset
`fd = open("file", O_RDONLY)`	3	0
`read(fd, buffer, 100)`	100	100
`read(fd, buffer, 100)`	100	200
`read(fd, buffer, 100)`	100	300
`read(fd, buffer, 100)`	0	300
`close(fd)`	0	–

Read past end of file returns **0**: the EOF signal

Jump with lseek(), then read

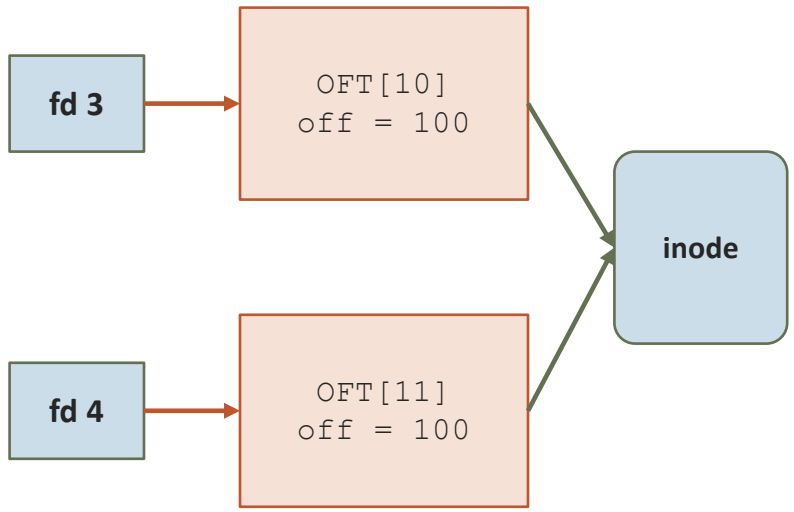
System call	Returns	Offset
`fd = open("file", O_RDONLY)`	3	0
`lseek(fd, 200, SEEK_SET)`	200	200
`read(fd, buffer, 50)`	50	250
`close(fd)`	0	–

Offset in the 300-byte file



Opening the Same File Twice

System call	Returns	OFT[10] off	OFT[11] off
`fd1 = open("file", O_RDONLY)`	3	0	–
`fd2 = open("file", O_RDONLY)`	4	0	0
`read(fd1, buffer1, 100)`	100	100	0
`read(fd2, buffer2, 100)`	100	100	100
`close(fd1)`	0	–	100
`close(fd2)`	0	–	–



Two entries, two **independent** offsets, one underlying file

Sharing Table Entries Across fork()

fork-seek.c (Figure 39.2)

```
int main(int argc, char *argv[]) {
  int fd = open("file.txt", O_RDONLY);
  assert(fd >= 0);
  int rc = fork();
  if (rc == 0) {              // child
    rc = lseek(fd, 10, SEEK_SET);
    printf("child: offset %d\n", rc);
  } else if (rc > 0) {        // parent
    (void) wait(NULL);
    printf("parent: offset %d\n",
           (int) lseek(fd, 0, SEEK_CUR));
  }
  return 0;
}
```

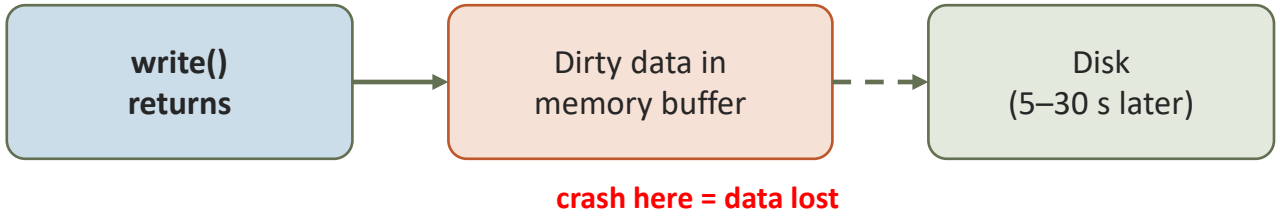
Output

```
prompt> ./fork-seek
child: offset 10
parent: offset 10
```

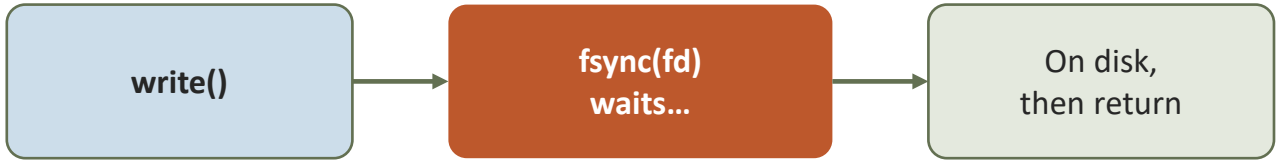
- Child's fds point to the **same** table entry
- Child's lseek() is visible to the parent
- Refcount = 2; entry freed after both close
- Use: cooperating processes write one shared output file without extra coordination

Forcing Writes to Disk with fsync()

Without fsync(): eventual persistence



With fsync(): forced now



Example

```
int fd = open("foo",
    O_CREAT|O_WRONLY|O_TRUNC,
    S_IRUSR|S_IWUSR);
assert(fd > -1);
int rc = write(fd, buffer, size);
assert(rc == size);
rc = fsync(fd);
assert(rc == 0);
```

- Needed by databases and other apps with recovery protocols
- New file? Also fsync() the **directory** that contains it

Renaming Files Atomically with rename()

- `mv foo bar` uses `rename(old, new)`
- **Atomic** w.r.t. crashes: the file has the old name or the new name, never in between
- Building block for atomic file updates

Editor pattern (error checks omitted)

```
int fd = open("foo.txt.tmp", O_WRONLY|O_CREAT|O_TRUNC,
              S_IRUSR|S_IWUSR);
write(fd, buffer, size); // new version
fsync(fd);
close(fd);
rename("foo.txt.tmp", "foo.txt");
```

1. Write new version to foo.txt.tmp
2. fsync(): new data is on disk
3. close() the temp file
4. rename(): atomic swap into foo.txt

Crash at any point: complete old file OR complete new file

Getting File Metadata with stat()

Figure 39.5: struct stat

```
struct stat {
  dev_t     st_dev;     // device of file
  ino_t     st_ino;     // inode number
  mode_t    st_mode;    // type + protection
  nlink_t   st_nlink;   // # hard links
  uid_t     st_uid;     // owner user ID
  gid_t     st_gid;     // owner group ID
  dev_t     st_rdev;    // device ID (special)
  off_t     st_size;    // size in bytes
  blksize_t st_blksize; // FS I/O block size
  blkcnt_t  st_blocks;  // blocks allocated
  time_t    st_atime;   // last access
  time_t    st_mtime;   // last modification
  time_t    st_ctime;   // last status change
};
```

Running example

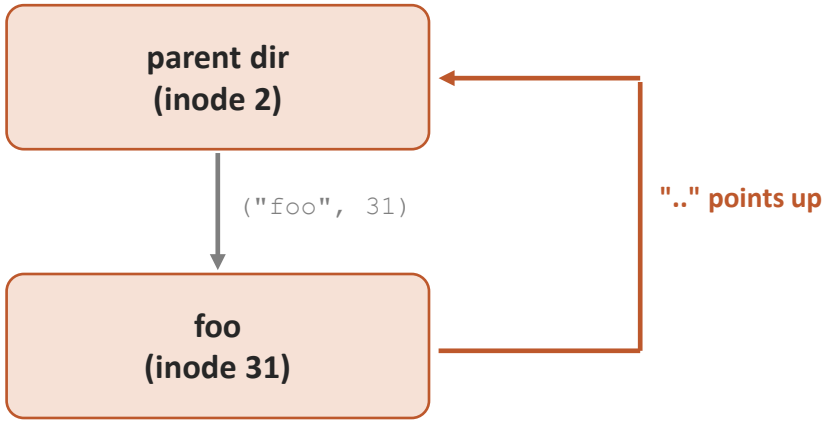
```
prompt> echo hello > file
prompt> stat file
  Size: 6   Blocks: 8
  Inode: 67158084
  Links: 1
  Access: (0640/-rw-r-----)
```

- Metadata: data **about** a file
- `stat(path)` / `fstat(fd)` fill a struct stat
- Stored in the file's **inode** on disk; active inodes cached in memory

Making Directories

- You can **never write a directory directly**: it is FS metadata
- Update it indirectly by creating files or subdirectories in it
- `mkdir()` system call (used by the `mkdir` program)
- An "empty" directory has two entries: `.` (itself) and `..` (parent)
- `cd ..` is just a lookup of the name ".."

```
prompt> strace mkdir foo
mkdir("foo", 0777) = 0
prompt> ls -a
./ ../
```



Contents of directory foo

Name	Inode	Meaning
.	31	this directory
..	2	parent directory

Reading and Deleting Directories

A tiny ls

```
int main(int argc, char *argv[]) {
  DIR *dp = opendir(".");
  assert(dp != NULL);
  struct dirent *d;
  while ((d = readdir(dp)) != NULL) {
    printf("%lu %s\n",
      (unsigned long) d->d_ino, d->d_name);
  }
  closedir(dp);
  return 0;
}
```

Each directory entry

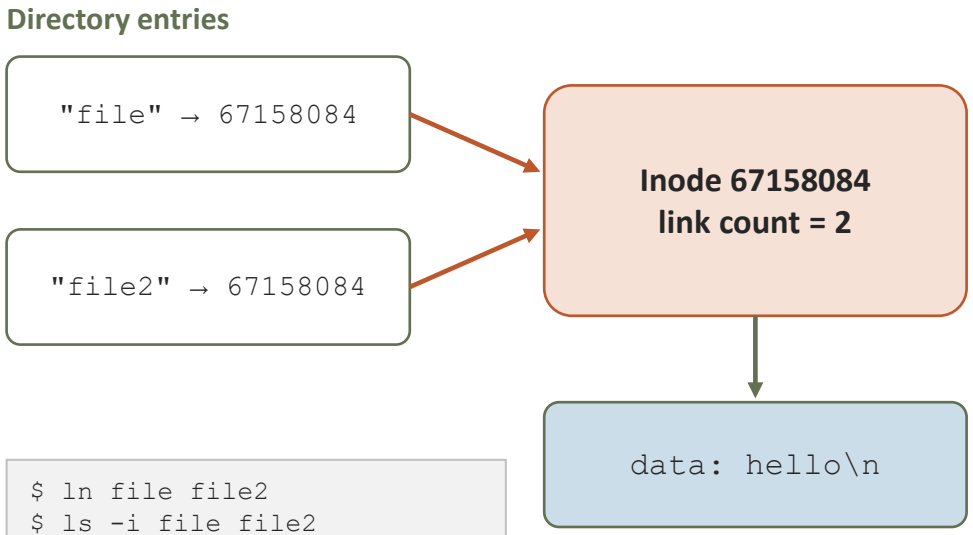
```
struct dirent {
  char  d_name[256]; // filename
  ino_t d_ino;       // inode number
  off_t d_off;       // next dirent
  unsigned short d_reclen;
  unsigned char  d_type;
};
```

- Directories hold little: name → inode number
- `ls -l` calls stat() on every entry
- `rmdir()` works only on an **empty** directory

Be wary of powerful commands: `rm -rf *` run from the root directory deletes the whole file system

Hard Links: Why Removing a File Is unlink()

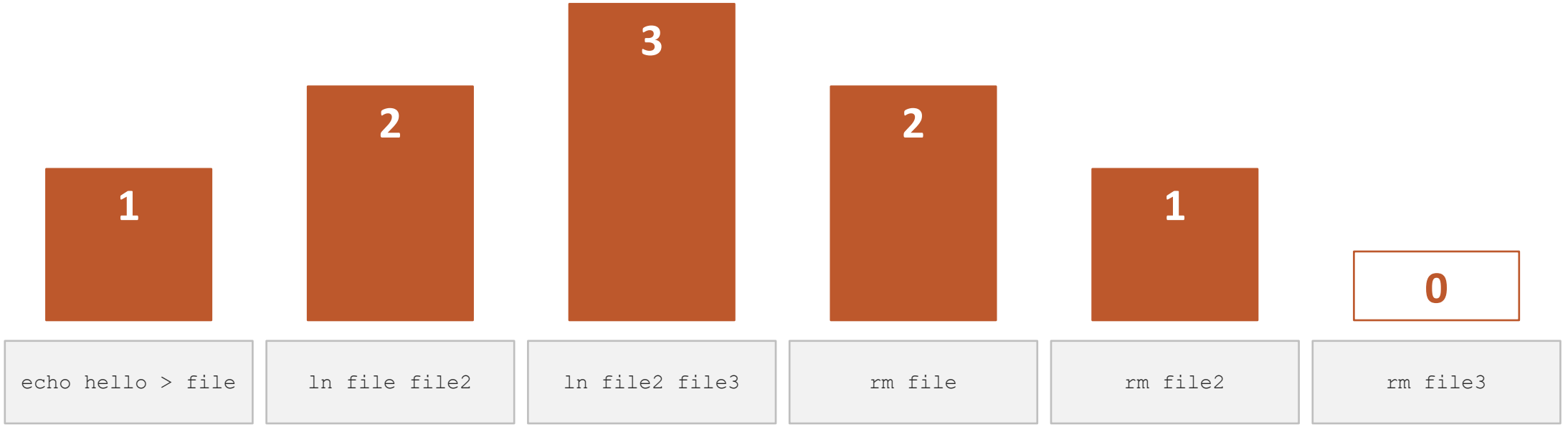
- `strace rm foo` shows `unlink("foo")`. Why not "delete"?
- `link(old, new)` / `ln file file2`: a new **name** for the **same inode**; no copy
- Creating a file = make an inode + link a name to it in a directory
- `unlink()` removes one name and decrements the **link count**
- Inode and data freed only when link count reaches 0




```
$ ln file file2
$ ls -i file file2
67158084 file
67158084 file2
$ rm file
$ cat file2
hello
```

Watching the Link Count

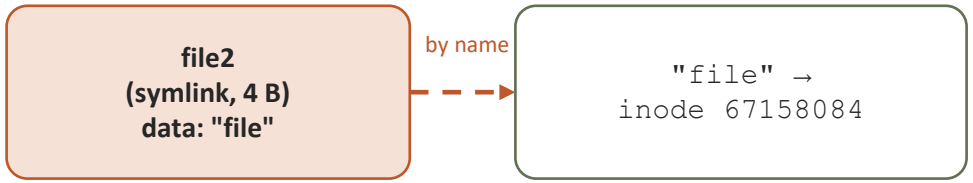
Links: value of st_nlink after each command (inode 67158084)



Count reaches 0 → file system frees the inode and its data blocks: the file is truly deleted

Symbolic (Soft) Links

- `ln -s file file2` creates a symbolic link
- A **third file type**: `l` in `ls -al`, "symbolic link" in stat
- Its data is the **target's pathname**: file2 is 4 bytes = len("file")
- Can link to directories and across file systems
- Dangling reference: remove the target and the link breaks




```
lrwxrwxrwx 1 remzi remzi 4 May 3 file2 -> file
-rw-r----- 1 remzi remzi 6 May 3 file
```

Dangling reference

```
$ ln -s file file2
$ rm file
$ cat file2
cat: file2: No such file or directory
```

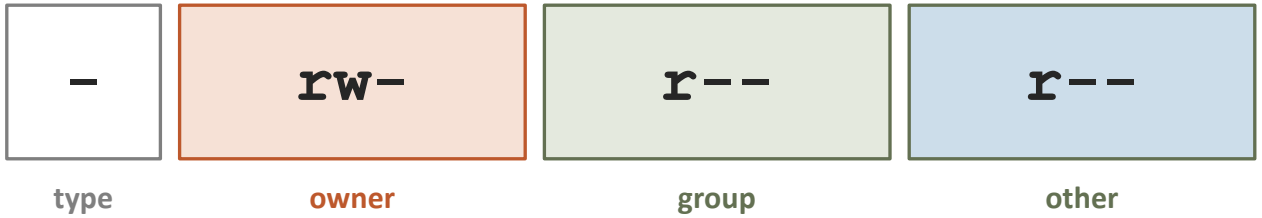
Hard Links vs Symbolic Links

Property	Hard link (`ln`)	Symbolic link (`ln -s`)
What it is	Another directory entry → same inode	A separate file whose data is a pathname
Own inode?	No: shares the target's inode	Yes: file type "symbolic link"
Link to a directory?	Not allowed (could create cycles)	Allowed
Across file systems?	No: inode numbers are per file system	Yes: it just stores a path
Original name removed	Data still reachable; link count drops	Link dangles: "No such file"
Seen in ls -l	Looks like a regular file	Starts with l; shows name -> target

Deleting a file = performing the **last unlink()** of it from the directory tree

Permission Bits

```
prompt> ls -l foo.txt
-rw-r--r-- 1 remzi wheel 0 Aug 24 16:29 foo.txt
```



r = 4, w = 2, x = 1 (OR the bits in each group)
`chmod 600 foo.txt`: owner 6 = 4+2 (rw-), group 0, other 0
Result: `rw-------`

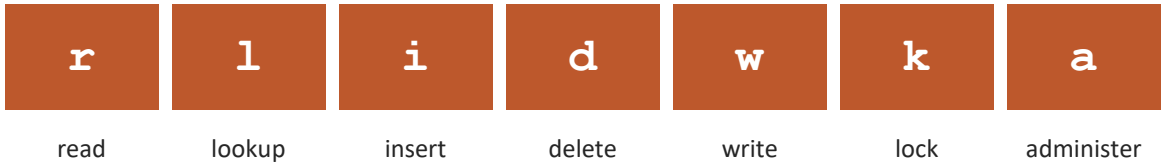
- Files are often **shared**, so FS needs access control
- Three classes × three rights (read, write, execute)
- Execute bit on a **file**: can it run?
  - `chmod 600 hello.csh` → "Permission denied"
- Execute bit on a **directory**: may cd into it; with w, may create files

Access Control Lists and the Superuser

AFS: one ACL per directory

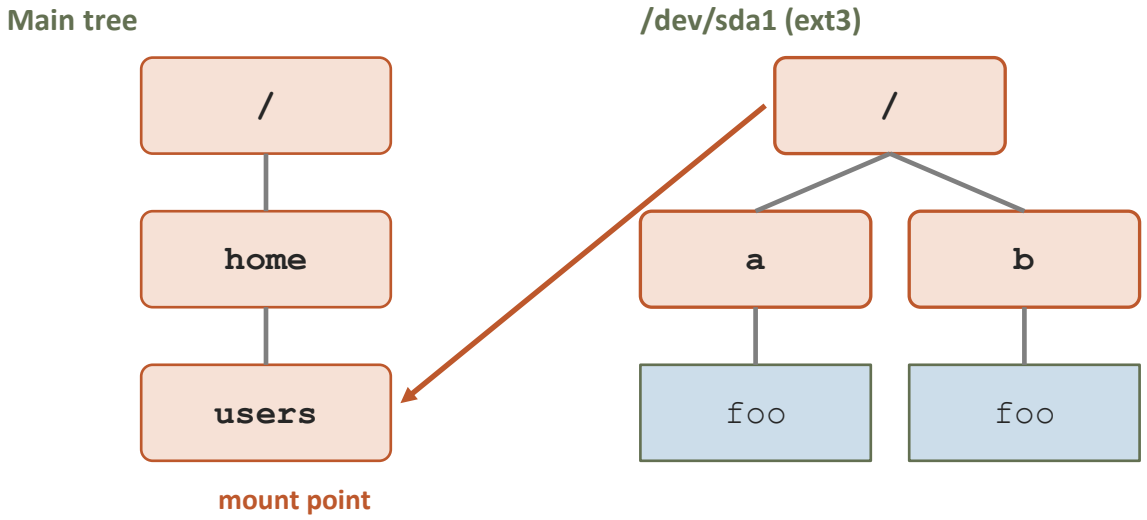
```
prompt> fs listacl private
Access list for private is
Normal rights:
  system:administrators rlidwka
  remzi rlidwka

prompt> fs setacl private/ andrea rl
```



- ACL: an explicit list of **who** may do **what**
- More precise than owner/group/other
- **Superuser** (root, or AFS system:administrators) can access all files
- Trusted users are an inherent security risk if impersonated

Making and Mounting a File System




```
prompt> mount -t ext3 /dev/sda1 /home/users
prompt> ls /home/users/
a b   # files: /home/users/a/foo, /home/users/b/foo
```

- `mkfs`: write an empty file system (with a root dir) onto a partition
- `mount`: paste it onto the tree at a **mount point**
- One uniform tree for many file systems

Output of mount (excerpt)

```
/dev/sda1 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
tmpfs on /dev/shm type tmpfs (rw)
AFS on /afs type afs (rw)
```

Summary: Key File System Terms

File	Directory	File descriptor	Open file table
Array of bytes; low-level name = i-number	(name, i-number) pairs, incl. `.` and `..`; forms a tree	Private per-process handle returned by open()	Entry holds file, current offset, access mode, refcount

Offset & lseek()	fsync()	Links	Permissions & ACLs
read/write advance it; lseek() moves it, no disk I/O	Force data (and the dir entry) to persistent media	Hard = same inode; soft = stored path; delete = last unlink()	rwx for owner/group/other; ACLs for finer control

Thank You!!